.SYSTEMX: Portable Project Coordination for AI-Assisted Work

White-paper edition 2.2 | Software baseline: 1.11.0-beta.4

Prepared 31 October 2026 by Lucas (SatoshiUNO) for Wayne Tech Lab LLC.

Source for this edition | Release report | Manual | Historical edition 2.1

Abstract

`.SYSTEMX` is a portable record and coordination layer for AI-assisted projects. It keeps an objective, master plan, canonical task ledger, selected memory, and verification evidence in a versioned project folder. The beta line also supplies optional role records, a default-deny Access Matrix, an unset Agenda settings catalog, named BOT/DOT workspaces, advisory security review, and bounded local command interfaces. A host may use these records with its own CLI or IDE, but the folder does not authenticate a worker, grant provider rights, install an operating-system service, or accept a task on the owner's behalf. The public repository is a reusable template; large authoring files, including Blender projects, belong in separately managed external data.

This edition documents the software carried into `1.11.0-beta.4`, including beta.2 governance and toolchain additions and the beta.3 current visual atlas. It records source and release evidence with explicit limits. It is not an independent security audit, malware-clean certificate, live provider interoperability result, cross-platform host acceptance report, or measured token-savings study. Editions 2.0 and 2.1 remain immutable historical papers for their own alpha.4 and beta.1 technical baselines.

1. One coordination layer, distinct authorities

The exact hidden directory name is `.SYSTEMX`. An adopted root keeps purpose and constraints in `GLOBAL/CONTEXT.md`, outcomes in `PLAN/MASTER-PLAN.md`, canonical task states in `WORK/TASKS.json`, and accepted decisions or resumable checkpoints in selected memory. A registered child project keeps its own records under `Projects/NAME/.SYSTEMXP` while sharing reviewed outer tools. The public template begins with blank project facts. Setup and updates are additive: a managed release keeps its own immutable defaults, previews conflicts, and preserves adopter-owned records.

Each independently accepted unit of work receives a stable task ID and explicit acceptance condition. The six task states are `todo`, `in_progress`, `blocked`, `needs_review`, `done`, and `cancelled`. Generated views and harness TODO lists are projections or external observations; they do not replace the canonical ledger. `done` depends on review against the task's criteria, not just an exit code or a worker's completed flag. Optional ROOT links request provenance and planning levels to observed harness or bot items using explicit bindings; its read-only delta identifies differences for an authorized actor to reconcile.

Agent 0 coordinates the selected scope; the user or designated owner retains acceptance. Host tools and provider accounts enforce permissions. A role label, bot name, roster row, or Access Matrix entry cannot grant access. Native subagents, CLIs, and online assistants retain separate host identities and require action-time readback.

Agent X records event time and due observations; it does not execute a scheduler. Agent Y handles an explicitly delegated installed CLI and a bounded branch/PR lane; a worktree does not isolate its operating-system or network access. Agent Z computes a repeatable score from supplied answers and evidence references; it does not perform every underlying check. Agent Alpha selects among ten AUTOBOT reference files on demand, rather than starting ten model processes. These roles supplement Agent 0 without displacing canonical task acceptance.

Agent Beta (`agent.b`) registers named offline workspaces under `AGENTB/.SYSTEMXB-BOT_NAME` with nonsecret metadata and a selected source such as `local` or `online-reference`. It does not contact the named service. Agent 1's 400-question catalog is separate from Agent Z's 100-question review policy. The read-only `root roster` combines selected-scope pointers to role registrations, named instances, task counts, provider aliases, and access-decision IDs. It paginates results and exposes a source revision. Its `identityVerified`, `accessGranted`, `runtimeStarted`, and `executed` fields remain false; a roster read neither activates a role nor starts a provider.

Agenda is an optional Agent Alpha settings table. The stock `AUTOBOT/AGENDA-SETTINGS.json` catalog defines ten context-selection settings, one per fixed AUTOBOT reference, with `unset` defaults; selected-scope `AUTOBOT/AGENDA.json` starts with an empty `options` array. A reviewed choice may be `include` or `exclude`. Inclusion requires a same-scope reviewed-intent decision and a current digest of the chosen Rules file. Agenda does not schedule work, run AUTOBOTs, change provider instructions, or grant access. `session-settings --compact` gives bounded status and an aggregate Rules revision so a session can load only relevant source text. The digest detects local change, not external truth or permission.

2. Access Matrix: complete role coverage, no implicit grant

`GLOBAL/ACCESS-MATRIX.json` is the machine-readable selected-scope record. The distributed `GLOBAL/ACCESS-MATRIX.xlsx` is a five-sheet human-reviewable stock view that includes an Agent 0 roster tab. The standard catalog has 20 rows: seven named agents (0, 1, Alpha, Beta, X, Y, Z); one generic numeric-worker pattern; one host-native subagent pattern; one Agent Beta BOT/DOT instance pattern; and ten fixed AUTOBOT reference roles. Every standard row has `templateGrant: none` and `requiresReview: true`. The stock `GLOBAL/ACCESS-DECISION-RULES.json` defines outcomes and required review fields. A new project's `decisions` list remains empty. No account name, credential, project authorization, or person-specific grant is seeded into the public template.

An applied access decision is a separate, selected-scope `ACC-...` record with an actor, resource, action, environment, owner, evidence, review time, and recheck context. The allowed outcomes are `reviewed-intent`, `denied`, and `gap`; reviewed intent requires a policy reference. Even reviewed intent is not a verified external grant. The format cannot enforce GitHub IAM, operating-system ACLs, an IDE's tool policy, or cloud authorization. A workbook edit alone is not an active decision: validate and reconcile the

canonical JSON before treating a row as current. A bot workspace may cite a decision ID, but a cited row still does not authenticate the bot or open a provider connection.

`agenda decision-add` previews one scoped decision and requires a matching revision plus `--apply` to write it. A changed revision rejects application. `agenda workbook-export` likewise requires preview, `--expected-revision`, and `--apply` to write a deterministic, formula-free six-sheet selected-scope view under ignored `local/`. `agenda workbook-check` compares it with canonical JSON. Neither workbook grants access.

3. Security review and secret delivery

Agent 1 (`agent.1`, when that numeric ID is free) defaults to warning-only `all-in-one` review. Its catalog contains 20 categories of 20 questions each and ten event buckets. The local check can inventory dependency manifest names and explicitly selected `.env` variable names without returning values. Unanswered questions remain unanswered; a filename scan is not a 400-question assessment. Device malware, spyware, keylogger, and MFA-interception posture are bounded signals requiring appropriate external evidence. Unknown, untested, and not-applicable states must not be reported as pass.

First run asks the operator to identify a personal or enterprise context, repository and role, credential consumer, target, local store, and rotation path. `env-inventory` and `env-sanitize` preview key names, duplicates, unsupported lines, Git tracking, and a names-only `.env.example`; an explicit apply creates the example without deleting the source. Newly issued credentials are the preferred migration path; an existing credential can be a bounded test choice while rotation is reviewed. The original `.env` and historical logs remain under the owner's retention and migration decision until consumers are verified.

GitHub Actions secrets are workflow inputs, not a local `.env` store; a symlink cannot deliver one to a local CLI or IDE. GitHub Agents, Codespaces, Dependabot, and Actions secrets have distinct consumers and scopes. The implemented `env-migrate` path targets repository or environment Actions secrets one name at a time through hidden terminal input after live account and administrator checks. It does not configure an OS credential store, organization secret, or local process. `gh secret list` confirms name and scope metadata, not the stored value or consumer behavior. This public repository has GitHub Actions disabled and ships no workflow; adopters choose their own policy. Repository writer access can permit workflow changes, so a `.SYSTEMX` guest label cannot itself protect Actions secrets. GitHub's Actions secret guidance describes the provider's scopes and CLI behavior.

4. CLI and IDE interoperability

First run and `onboard plan` expose an unset catalog of project profiles, provider surfaces, account contexts, and credential consumers. Selecting a catalog entry returns setup guidance; it does not install, sign in, connect, or execute a provider. The managed `systemx run --target PATH --offline -- ...` route verifies a selected installation and invokes its commands. `--offline` suppresses the manager's optional release check; it is not a network sandbox for other programs. A registered child requires explicit `--project NAME` selection. The optional root `AGENTS.md` bridge previews a short pointer to the canonical guide and preserves an existing root instruction file for review.

The host-owned `service start --stdio` process exposes fixed read-only JSON-lines methods: `ping`, `capabilities`, `assess`, `roster`, `settings`, `onboard`, and `stop`. It accepts no per-request path, provider, account, secret, shell command, or write argument. One session handles at most 64 requests, with a 16 KiB input-line and 512 KiB response bound. It listens on no port and installs no daemon or native IDE extension. The process inherits the launching user's OS rights. Its responses may contain project names, task metadata, local paths, and hashes; review them before forwarding them to an external model. Reviewed changes use direct CLI preview and apply operations. A successful protocol response is not a security verdict.

VS Code, Codex CLI or SDK, Cursor, Kiro, and JetBrains can use a host-controlled terminal or subprocess to read this contract. This beta does not ship a native VS Code extension, Codex tool registration, MCP server, ACP agent, or direct chat-panel binding. Those host protocols need separate adapters and platform tests. The provider catalog separates Codex CLI, IDE, SDK, and OpenAI dots; Grok Build CLI and Grok Bot; Muse Code and personal Muse; Kiro CLI, IDE, and Web; and AWS CLI infrastructure accounts. The named built-in Agent Y task adapters cover Codex CLI, GitHub Copilot CLI, and Muse Code; a generic configured CLI adapter is also available after task-specific review. Muse Code's adapter is source-tested; live account and fresh-worktree trust remain unverified. Grok Build, Kiro CLI, Claude Code, OpenCode, BLACKBOX AI CLI, Gemini CLI, Cursor CLI, and Aider remain manual discovery choices until their execution contracts are configured and reviewed. Provider documentation governs sign-in, tool approval, cost, and repository access.

5. Release identity, logs, and deployment accounting

The beta continues the published `1.11.0-alpha.N` line as `1.11.0-beta.4`; resetting it to `0.0.01` would obscure the existing release history. The publication tag for this edition is `v1.11.0-beta.4`, and Python package metadata uses `1.11.0b4`. `VERSION`, `SOURCE.json`, package metadata, and the exact-file distribution manifest must agree at publication. The manifest records SHA-256 hashes for portable template files. Published tags and releases are immutable identities; later fixes need a new version rather than moving a tag. These conventions follow Semantic Versioning 2.0.0 and the Python packaging version specification.

A release counter counts published software release records under a stated selection rule and query time. A deployment counter reports observed GitHub deployment records separately, not Git commits, tags, packages, screenshots, or successful local tests. Failed or incomplete provider reads return `unknown`, never a fabricated zero. A deployment record does not establish a successful or live service. The template may be released without deploying any downstream application. Keep release, source validation, package, provider, installed-host, environment, and owner-acceptance evidence separate in logs and release notes. GitHub's deployment history describes the provider record; downstream providers require their own receipts.

6. Template, current media, and external data

The public repository distributes portable defaults, current images, documentation, code, tests, and a file inventory. It must not ship a real project registry, account, secret, absolute private-drive path, owner payload, raw render cache, or editable Blender project. The current media library contains exactly twelve canonical SVGs: four system maps, seven detailed agent guides, and one atlas assembled from the eleven final panels. Each guide gives folder paths, example prompts, CLI routes, process flows, and authority limits; a reading-order text alternative accompanies them. The SVGs declare a 7680 x 4320 canvas with selectable vector labels. Their generated illustrations came from smaller 1672 x 941 masters, so the separate 8K PNG export pack does not imply native 8K raster detail. The atlas is a navigational overview; small details belong in the individual guides and text alternative. Their presence does not imply that a service, daemon, or bot ran.

The portable source contains one current graphic per topic and no video, raw generated master, editable Blender project, narration, or render cache. Earlier graphics remain readable in immutable beta.2 history and separately verified owner archives; edition 2.1 retains its original beta.1 publication identity. Selected source art and prompts are held outside the code distribution. The package and managed-release archive have finite size limits; large authoring files would increase installation cost. Derived media outside the package needs a release identity and checksum, while selected graphics appear in the source manifest. A Git ignore rule helps prevent accidental inclusion but does not prove that material was never committed or synchronized elsewhere. An external path reference is valid only when an authorized user can actually access that location in the selected environment.

7. Input efficiency and what remains unmeasured

The intended efficiency path is to read selected scope, a bounded task/context packet, the few relevant Rules files, and evidence on demand. A `session-settings --compact` response omits repetitive per-rule hashes from the full output and replaces them with a single aggregate revision. A development sample of the blank beta.1 template returned 3,248 bytes in the full form and 1,043 bytes in the compact form, a 67.9% output-byte reduction for that command. The exact byte count is sample-specific and can change with records or formatting. It is not a measured reduction in billed input tokens, total latency, total cost, or accepted-work effort.

To evaluate real savings, compare representative tasks with the same model, tools, acceptance criteria, and cache conditions. Record all workers' provider-reported input, cached input, output, tool calls, retries, elapsed time, and accepted outcome. Include the cost of creating and maintaining the handoff and correcting omitted context. A shorter packet that omits a necessary constraint is a regression, regardless of its byte count. No controlled cross-provider benchmark or production cost study is claimed in this edition.

8. Beta acceptance boundary and next research

The beta.3 release report records the twelve-file media verifier, source-bound atlas and export manifest, a clean wheel-to-manifest gate, installed-wheel smoke checks, and Python suites on 3.11.15 and 3.14.5. Those observations are tied to beta.3; the accompanying beta.4 release report records the final beta.4 source revision and its own qualification receipts. A reusable source distribution can be accepted after selected-scope setup, validator, managed update, build, exact wheel payload, workbook/JSON parity, media inventory, and public links are checked against one final revision. That is a source-and-package result. A downstream installation needs its own account verification, secret-consumer test, provider and host integration test, security review, deployment receipt when applicable, and owner acceptance. Agent 1's helper remains manual stdio. Windows and Linux native host integration, live Muse execution, and an actual OS credential consumer require separate evidence. The 400-question instrument starts with zero answered questions; its existence is not a security certification.

The next useful evaluations are task-level token and acceptance comparisons, tested adapters for chosen host protocols, and end-to-end credential-consumer checks on consenting adopter fixtures. An adopter should initialize a selected project, review its actual accounts and resources, record narrowly scoped access decisions, then test the provider and runtime that will consume them. Keep warnings advisory unless the project owner adopts a stricter policy. Preserve prior release, log, visual, and white-paper identities as historical evidence. Edition 2.0 remains an alpha.4 account and edition 2.1 remains the beta.1 account; this edition describes the current beta line with the limits above.

Source notes

1. WayneTechLab, `.SYSTEMX` source for edition 2.2, especially `VERSION`, `SOURCE.json`, `config/distribution.json`, `GLOBAL/ACCESS-MATRIX.json`, `GLOBAL/ACCESS-DECISION-RULES.json`, `AUTOBOT/AGENDA-SETTINGS.json`, and selected-scope CLI scripts. The published tag and release report identify the final source and checks; this paper does not claim independent validation.

2. WayneTechLab, beta.3 release report, visual asset manifest, and text alternative. The media and test figures in Section 8 are tied to that immutable release.

3. WayneTechLab, Agent 0 roster, direct I/O, first run, and Access Matrix guidance, describing implemented routes and their authority limits.

4. WayneTechLab, edition 2.1 white paper and edition 2.0 history. Earlier papers remain evidence for their own technical baselines.

5. Semantic Versioning 2.0.0 and Python Packaging Authority, Version specifiers, for prerelease version identity and Python spelling.

6. GitHub, Using secrets in GitHub Actions and Viewing deployment history, for provider-specific secret and deployment boundaries.